

Guess Market - Exercise 2

Submitter details

Name	Stav Hecht
ID number	211794151
Email	stavhe@mta.ac.il

Project repository

<https://github.com/stavhecht/GuessMarket>

Bonuses implemented

Saving and loading the system state (Java serialization). The desktop UI has *Save session / Load session* in the file bar; the console keeps them as menu options 8 and 9. The user gives a full path including the file name **without an extension**; `.gm` is appended by the engine. (fits mac and windows OS, note that windows path is with `"\"` so when asked to write full path to saved session it need to look like `"\\PATH\\your_desired_file_name"`). A saved session also carries the window's own state, so reopening it restores the theme and the preferences it was saved with.

Three colour themes. Light, Dark and Neon, switched live from the button in the file bar. A theme is one block of looked-up colours in `guessmarket.css`; `Theme.applyTo` swaps a single style class on the root (`theme-light` / `theme-dark` / `theme-neon`) rather than loading a second stylesheet. Neon changes the type as well as the palette.

Animations. Every figure that carries money or a price is a rolling counter (`Ticker`) rather than a number that is overwritten: it counts to its new value when the value really moved, and lands instantly when it is pointed at a different event or user, so switching selection never animates a trade that never happened. The load bar ramps and the green LOADED tick holds for four seconds. All of it can be turned off from the *animations* check box on the tab strip, which gates the animation only: every figure is still written, so an unticked window is never stale, only still.

Live charts. Both screens draw a `SparkChart`: the Events screen plots the two option prices over the life of the market, the Users screen plots the selected account's balance. Every point is a dot and hovering one names the series, the value and the moment it happened. Both series **begin at the market's opening point**, 0.5 a side for LMSR (the scoring rule at zero shares) and $d/2$ for an order book, and the balance timeline begins at the account's initial cash, so a chart shows where the market came from rather than only what has happened since the window opened.

Creating an event from inside the application. *Create event* on the Events screen opens a form for either trading method, and **whoever creates an event is its Market Maker**: the acting user pays for it ($b \cdot \ln 2$ for LMSR, `initial` for an order book), and is the only user who can close it. The engine checks

their current balance covers it before anything is built, so a market is never opened on money that is not there.

Build and run

Nothing has to be installed but a JDK. Every dependency is in the folder beside `UI.jar`, so any of these starts the application:

```
java -jar UI.jar      # from the folder itself, on any platform
run.bat              # the same on Windows
./run.sh             # the same on macOS or Linux
```

Two sets of dependencies, neither optional, and both named in the jar's manifest `Class-Path`:

- the **JAXB jars**, since `jakarta.xml.bind` left the JDK in Java 11.
- **JavaFX** (`base`, `graphics`, `controls`, `fxml`), which is not in the JDK either. `javafx.fxml` is among them because the window's shell is loaded from `DesktopApp.fxml`. These four are **not the SDK's jars as they come**: they are merged from the macOS arm64 and Windows x64 SDKs of the same 25.0.4 build, with both platforms' native libraries packed inside `javafx.graphics.jar`, which JavaFX extracts by platform-specific name at first start. One set of jars therefore runs on either platform with no SDK, no module path and no VM options.

Every start prints one `WARNING: Unsupported JavaFX configuration` line, which is expected when JavaFX is on the class path rather than the module path and affects nothing.

Requires **JDK 17+** (records, sealed interfaces, switch expressions, `@Serial`).

Two front ends, chosen in `ui.Main` by the `exN` variable: 1 runs the exercise-1 console menu, 2 runs the JavaFX desktop application. Both drive the same `MarketEngine`.

Main classes

Class	Role
<code>MarketEngine</code>	Façade - the only engine class either UI imports, and the only place DTOs are built. One method per user command.
<code>EventManager</code>	Owns the events (<code>LinkedHashMap</code> , file order) and the users. Holds the entirety of the mutable state, which is what makes save/load one <code>writeObject</code> .
<code>Event</code>	Aggregate root - owns its two <code>Options</code> , its <code>Account</code> , its <code>OrderBook</code> and its trade history. All state changes go through it.
<code>TradingMethod</code>	Sealed interface, one record per method: <code>Lmsr(b)</code> and <code>OrderBook(allowMint, initialInvestment, d)</code> . Decides which half of the engine an event uses.

Class	Role
LmsrCalculator	The market-maker maths: prices, cost (C(q)), purchaseCost, initialSubsidy. Pure and stateless.
OrderExecutor	Where a fill happens: matching, minting, money between two users and the event account. Stateless; everything arrives as an argument.
OrderBook / Order / BookTrade	The queues, the sequence numbers and the price indicators. Bookkeeping only, no money.
XmlEventLoader	File → domain: unmarshal via JAXB, apply every rule about a file, build Events.
engine.schema	JAXB classes describing the file format only, element names copied letter for letter. No domain logic.
ConsoleApp / InputReader / OutputFormatter	The terminal front end: menu and dispatch; the only class that reads the console; the only class that writes to it.
DesktopApp	The JavaFX front end: loads DesktopApp.fxml as the window's shell, owns the file bar, the tabs and the status line.
EventsScreen / UsersScreen	The two tab bodies, generated in Java because most of a screen comes from the loaded market.
LmsrPane / OrderBookPane / CreateEventDialog	The two trading panels and the create-event form.
LiveMarket / MarketData	The adapter that turns a polled engine into properties the UI can listen to, and the read-only derivations the screens draw.
SparkChart / Ticker / Widgets / Theme	The chart canvas, the rolling counters, the shared controls and formatting, and the theme tokens read out of guessmarket.css.

Choices made

Failure is atomic by construction, not by try/catch. The loader throws before a single object reaches `EventManager`, so a rejected file leaves the previous session intact. `loadState` reads into a local and reassigns its field only after the invariants hold on it. `participate` validates and computes everything *before* the first mutation. `OrderExecutor` cannot rehearse a fill, because what it costs depends on the orders it meets, so it validates the submitter against the **worst case** instead, which is what lets every step after that commit as it goes.

One exception hierarchy, caught in one place. Everything a user can cause extends `EngineException` (unchecked). `ConsoleApp.dispatch` catches it exactly once and prints the message, `DesktopApp.perform` paints it in the status bar; the individual handlers contain no try/catch at all. The one deliberate exception is `CreateEventDialog`, which is a form rather than a handler: it calls the engine inside a filter on its own Create button, so a refusal paints the message on the form instead of closing a window full of typing.

DTOs are constructed only in MarketEngine, never by a UI. `engine.dto` separates *operation outcomes* (`PurchaseResult`, `OrderResult`, `SettlementResult`) from *snapshots* (`EventView`, `EventStatusView`, `UserView`), which is why neither front end has to reach into the model to draw a

screen, and why the two front ends can look nothing alike while sharing every rule.